

Métodos Computacionais em Física de Altas Energias

Marco Dias

F056

6 de agosto de 2026

- Curso prático de computação em física de altas energias.
- Trabalhos semanais na sexta-feira
- Links marcados com cor ciano.

Conteúdo programático:

- Revisão de Unix, latex, makefile, git
- Python
- C++
- ROOT
- Combinação de linguagens de programação
- Geradores de eventos
- Simulação de detector e reconstrução
- Estatística Básica
- Computação em grade

O Shell

O Shell é um programa que executa automaticamente quando você faz login em um sistema Unix/Linux. O Shell forma a interface entre usuários e o resto do sistema. Ele lê cada comando digitado no terminal e interpreta o que foi solicitado.

Características:

- Variáveis
- Estruturas de controle (if, while, etc.)
- Sub-rotinas
- Passagem de parâmetros
- Tratamento de interrupções

Nome do Shell	Histórico
sh (Bourne)	O shell original
csh, tcsh, zsh	O C shell
bash	Bourne Again Shell, do projeto GNU

Redirecionamento e Pipes

Redirecionamento de saída com o símbolo `>`:

```
$ ls > myfiles
```

Redirecionamento de entrada com o símbolo `<`:

```
$ wc -l < myfiles
```

Pipes - A saída padrão de um processo pode ser a entrada padrão de outro processo. O operador pipe é o símbolo `|`.

Exemplos:

```
$ ls | sort  
$ ls | sort | more  
$ ls | sort | grep btag | more
```

Executando Scripts

Suponha que você tenha um arquivo chamado 'test' contendo comandos Unix.

Formas de executar:

```
$ csh test    (ou sh test)
$ source test
```

Ou dar permissões de execução ao arquivo:

```
$ chmod 755 test
$ test
```

O código numérico no comando chmod vem de `rwrxwrxrwx` com `rw` sendo número binário: `rw` $x=111=7$ e `r-x` $=101=5$.

Um script shell começa com uma linha indicando qual programa executa o arquivo:

```
#!/bin/sh
```

Comentários começam com `#` e vão até o final da linha.

Variáveis no csh

Tipos de variáveis:

- Variáveis de shell - definidas localmente no shell
- Variáveis de ambiente - definidas para o shell e todos os processos filhos

Comando set:

```
set
set name
set name = word
set name = (wordlist)
set name[index] = word
```

Comando setenv:

```
setenv
setenv name value
```

Para obter o valor de uma variável, use a referência `${name}`.

Exemplo de script csh

```
#!/bin/csh

if( ! ${?ENVIRONMENT} ) setenv ENVIRONMENT INTERACTIVE
if( $ENVIRONMENT != "BATCH" ) then
    if( ! ${?SCRATCH} ) then
        echo Setting workdir to HOME
        setenv WORKDIR $HOME
    else
        echo Setting workdir to SCRATCH
        setenv WORKDIR $SCRATCH
    endif
    setenv LS_SUBCWD $PWD
endif
setenv INPUTFILE analysis.in
setenv DATAPATH $LS_SUBCWD/data

cd $WORKDIR
if( -e $INPUTFILE ) rm $INPUTFILE
cp $LS_SUBCWD/$INPUTFILE $WORKDIR
setenv LD_LIBRARY_PATH $LS_SUBCWD/../../lib:$LD_LIBRARY_PATH

echo "START EXECUTION OF JOB"
```

No bash, a atribuição padrão do Bourne shell é usada:

```
name = value
```

Variáveis de array podem ser definidas de duas formas:

```
name[index] = value  
name = (value1, value2, ..., valueN)
```

Exportando variáveis para o ambiente:

```
export name  
export name = value
```

Avaliação aritmética:

```
$(( expression ))
```

Estrutura básica do if:

```
if condition ; then [comando 1] ; else [comando 2];fi
```


Estruturas de repetição:

```
for ((i=1; i<=5; i++)); do  
    echo "N mero: $i"  
done
```

```
contador=3  
while [ $contador -gt 0 ]; do  
    echo "Faltam $contador segundos"  
    ((contador--))  
done
```

Em experimentos de Física de Altas Energias, o software é tipicamente escrito pelos membros da colaboração.

CVS é uma ferramenta para gerenciar o código fonte:

- Previne sobrescrita de alterações de outros
- Mantém registro das versões
- Permite acesso fácil a versões e pré-versões

Documentação: <http://ximbiot.com/cvs>

Subversion (SVN) - outro sistema de controle de versão amplamente utilizado. Exemplo: LHC Higgs XS WG usa SVN. <http://subversion.apache.org>

Git - Sistema de controle de versão distribuído open source. Projetos que usam Git: Git, Linux Kernel, Perl, Ruby on Rails, Android, etc.

Git: Clonagem

Para obter uma cópia de um repositório Git existente:

```
git clone [url]
```

O Git recebe uma cópia completa de quase todos os dados que o servidor possui.

Exemplo:

```
git clone https://github.com/dune-mdias/F056
```

Cria um diretório chamado 'F056', inicializa um diretório .git dentro dele e faz o checkout da versão mais recente.

Para clonar com nome diferente:

```
git clone <url> novoNomeParaDir
```

Protocolos de transferência: https://, git://, ssh (user@server:path/to/repo.git)

Em máquina local:

```
git clone ~mdias/public/html/Test.git
```

Git: Estados dos Arquivos

Cada arquivo no diretório de trabalho pode estar em um de dois estados:

- **Rastreado (tracked)** - arquivos que estavam no último snapshot (podem estar não modificados, modificados ou staged)
- **Não rastreado (untracked)** - todos os outros arquivos

Ciclo:

- 1 Editar arquivos → modificados
- 2 Stage dos arquivos modificados
- 3 Commit das alterações staged

Comando principal:

```
git status
```

Para começar a rastrear um novo arquivo:

```
git add
```

Observação: Git add é um comando multi-uso - iniciar rastreamento, stage de arquivos

Git: Diff e Commit

Para ver exatamente o que foi alterado:

```
git diff
```

Para ver o que está staged para o próximo commit:

```
git diff --staged
```

Commit:

```
git commit -a -m 'Mensagem do commit'
```

É **obrigatório** adicionar uma mensagem de log.

Para commitar um único arquivo:

```
git commit meuArquivo -m 'Mensagem do commit'
```

Git: Movendo e Removendo

Para remover um arquivo do Git:

```
git rm arquivoObsoleto
```

Isso remove o arquivo do staging area e também do diretório de trabalho.

Para renomear um arquivo no Git:

```
git mv nomeAntigo nomeNovo
```

Visualizando histórico:

```
git log
```

Git: Desfazendo

Para corrigir um commit (commit muito cedo, arquivos esquecidos, mensagem errada):

```
git commit --amend
```

Exemplo:

```
git commit -m 'commit inicial'
git add arquivoEsquecido
git commit --amend
```

Resulta em um único commit.

Para descartar alterações em um arquivo:

```
git checkout arquivoModificado
```

Atenção: git checkout [arquivo] é um comando **perigoso**. As alterações são perdidas permanentemente.

Git: Repositórios Remotos

Repositórios remotos são versões do projeto hospedadas na Internet ou rede.

Exemplo: Criando um novo repositório no github e inserindo pela primeira vez:

```
# Initialize the folder as a Git repository
git init
# Stage all files in your folder
git add .
# Create your first local commit
git commit -m "Initial commit"
# Rename your default branch to main (recommended)
git branch -M main
# Link your local repository to GitHub (paste your copied URL)
git remote add origin [remote-name]
# Push your code to GitHub and set the default upstream branch
git push -u origin main
```


Logo depois, com o repositório já inicializado

```
git add .  
git commit -m "Minha Mensagem"  
git push # Upload your changes to GitHub
```

Git: Fetch e Push

Para obter dados de projetos remotos:

```
git fetch [remote-name]
```

Exemplo com merge:

```
git fetch origin  
git merge origin/master
```

Para compartilhar mudanças (push):

```
git push public
```

Primeira vez:

```
git push public refs/heads/master:refs/heads/master
```

Branching significa divergir da linha principal de desenvolvimento.
Um branch no Git é simplesmente um ponteiro leve e móvel para commits.

Criando um novo branch:

```
git branch test
```

Visualizando histórico com branches:

```
git log --oneline --decorate --graph --all
```

Git: Merging

Quando o desenvolvimento no branch 'test' está pronto, mescle com o master:

```
git checkout master  
git merge test
```

O Git cria um novo snapshot e um commit de merge (com mais de um pai).

Conflitos de merge: Quando dois branches têm mudanças na mesma parte do código, o Git não consegue mesclar automaticamente.

```
git status      # mostra conflitos como unmerged  
# Editar arquivos conflitantes manualmente  
git add arquivoConflitante  
git commit      # mensagem automaticamente criada
```

LaTeX é a ferramenta para escrever artigos científicos.

Documentação disponível em:

- <http://www.ctan.org/tex-archive/info/lshort/english/lshort.pdf>
- <https://xml.web.cern.ch/textproc.html>

Ferramentas úteis:

- 1 **BibTeX** - Gerenciamento de referências bibliográficas
- 2 **feynMF** - Desenho de diagramas de Feynman

BibTeX:

```
@Article{L1_TDR,  
  title = "The Level-1 Trigger",  
  journal = "CERN/LHCC 2000-038",  
  volume = "CMS TDR 6.1",  
  year = "2000"  
}
```

Citação: `\cite{L1_TDR}`

No arquivo tex:

```
\bibliographystyle{lesHouches}
```

Execução: latex → bibtex → latex → latex → dvips

feynMF:

```
\usepackage{feynmf}  
\begin{fmffile}{<METAFONT-filename>}
```

```
\begin{fmffile}{triangle_loop}
\begin{figure}[h]
\centering
\parbox{50mm}{
\begin{fmfchar*}(100,60)
  \fmfleft{i1,i2}
  \fmfright{o1}
  \fmf{curly,tension=0.5}{i1,v1}
  \fmf{curly,tension=0.5}{i2,v2}
  \fmf{fermion,tension=0.1}{v2,v1}
  \fmf{fermion,tension=0.3}{v1,v3}
  \fmf{fermion,tension=0.3}{v3,v2}
  \fmf{dashes}{v3,o1}
  \put{0,60}{small g}
  \put{0.5,0}{small g}
  \put{100,28}{small h,H,A}
\end{fmfchar*}
}
\end{figure}
\end{fmffile}
```

Make: Introdução

O utilitário make determina automaticamente quais partes de um programa precisam ser recompiladas e emite comandos para recompilá-las.

Arquivo makefile: Contém as regras que o make executa.

Nomes padrão: GNUmakefile, makefile, Makefile (nesta ordem).

Para usar nome diferente:

```
make -f myMakefile
```

Documentação: <http://www.gnu.org/software/make/manual/make.html>

Linhas em um makefile:

- Linhas de target
- Linhas de comando shell
- Linhas de macro
- Linhas de diretiva make (como include)

Make: Regras

Estrutura:

```
target: dependencies
    comando shell
```

Exemplo:

```
clean:
    rm -f *.o
```

Execução: \$ make clean

Importante: O comando shell deve ter um **tab** na frente. Se substituído por espaços, não funciona!

Dependências:

```
foo.o: foo.cc
    g++ -c foo.cc
```

Se foo.cc for mais recente que foo.o, o target é refeito.

Make: Regras Genéricas

Regra de sufixo:

```
.cc.o:  
    g++ -c $<  
  
main: a.o b.o c.o d.o  
    g++ a.o b.o c.o d.o
```

\$< é uma macro especial que substitui o arquivo fonte atual.

Macros:

```
OBJECTS = a.o b.o c.o d.o  
  
main: $(OBJECTS)  
    g++ $(OBJECTS)
```

Sintaxe de chamada de função: `$(function args)`

Exemplo usando wildcard, basename e addprefix:

```
files = $(wildcard *.cc)
filebasenames = $(basename $(files))
OBJECTS = $(addprefix .o, $(filebasenames))
```

- wildcard - lista arquivos no diretório local
- basename - remove sufixo
- addprefix - adiciona prefixo

Exemplo de Makefile completo:

```
files = $(wildcard ../src/*.cc)
OPT = -O -Wall -fPIC -D_REENTRANT
INC = -I$(ROOTSYS)/include -I. -I../src
LIBS = -L$(ROOTSYS)/lib -lCore -lCint -lHist

.cc.o:
    $(CXX) $(OPT) $(INC) -c $<

all: @$(MAKE) --no-print-directory All

All: @$(MAKE) compile && $(MAKE) analysis.exe

compile: $(OBJS)

analysis.exe: $(OBJS)
    $(CXX) $(LIBS) $(OBJS) -o analysis.exe

clean:
    rm -f $(OBJS)
```

Sed é um editor de fluxo (stream editor) para transformações básicas de texto em um fluxo de entrada.

Uso básico:

```
sed 's/hello/world/' input.txt
```

Substitui 'hello' por 'world' no arquivo input.txt.

Para modificar o arquivo diretamente:

```
sed -i 's/hello/world/' input.txt
```

Exemplo prático:

```
package:
  wget http://url.to/thepackage.tgz
  tar xfz thepackage.tgz
  cd thepackage
  sed -i '/CFLAGS/ s/-Wall/-Wall -fPIC/' configure
  ./configure
  make
```

Se o arquivo configure define flags em apenas um lugar:

```
sed -i 's/-Wall/-Wall -fPIC/' configure
```